操作系统实验 Lab 0 实验报告

姓名：范骐鸣

学号：24302010005

实验一：标准输出重定向 log_stdout

实现思路

实验一实现 xv6 用户命令 log_stdout i。程序把标准输入保存到缓冲区，并将标准输出重定向到以参数 i 命名的日志文件 i.log。

read_stdin 使用文件描述符 0 读取标准输入。全局缓冲区长度为 1024 字节，因此最多读取 1023 字节，最后一个字节保存 C 字符串结束符。程序使用 total 记录已经读取的长度，每次从 buf + total 开始写入，防止覆盖此前读到的数据。read 返回 0 表示到达 EOF，返回负数表示读取失败。

log_stdout 首先把 uint 类型的参数转换成十进制字符。程序通过反复除以 10 计算数字位数，再通过取模运算从低位到高位取得数字，并从字符数组右侧向左写入，最后追加 ".log"。参数为 0 时直接使用初始文件名 "0.log"。

xv6 中，文件描述符 0、1、2 分别表示标准输入、标准输出和标准错误。程序先调用 close(1) 释放标准输出，再打开日志文件。open 会分配当前最小的空闲文件描述符，因此日志文件会占用文件描述符 1。之后 printf 的内容便会写入日志文件，错误信息仍通过文件描述符 2 显示在终端。

最后，在 Makefile 的 UPROGS 中加入：

```
$U/_log_stdout
```

这样构建系统会编译 user/log_stdout.c，并将生成的程序写入 xv6 的 fs.img 文件系统镜像。

测试结果

执行 make fs.img 后，log_stdout.c 成功通过 RISC-V GCC 编译和链接，并被写入 fs.img。随后通过 make qemu 进入 xv6 测试。

实际终端记录：

```
$ echo hello | log_stdout 1
$ cat 1.log
hello
```

实验二：进程流水线 composites

实现思路

实验二要求使用 fork 和 pipe 递归构造进程流水线，对 2 ~ 35 进行分类。每一级子进程负责一个质数：能被该质数整除的数字写入当前进程的日志，不能整除的数字通过管道传递给下一级。

整体数据流如下：

```
主进程
生成 2～35
```

```
    |
    ▼
子进程 0：负责 2，输出到 0.log
    |
    | 不能被 2 整除的数
    ▼
子进程 1：负责 3，输出到 1.log
    |
    | 不能被 3 整除的数
    ▼
子进程 2：负责 5，输出到 2.log
    |
    | 不能被 5 整除的数
    ▼
子进程 3：负责 7，输出到 3.log
    |
   ...
```

main 首先创建第一条管道并 fork 出第 0 个子进程，然后将整数 2 ~ 35 依次写入管道。写完后关闭管道写端，使子进程在读完数据后能够收到 EOF。主进程最后通过 wait 等待第 0 个子进程结束。

sub_process 从左侧管道读取第一个数字，并将它作为当前进程的 factor。这个数字此前已经通过所有更小质数的筛选，因此必然是质数。当前进程调用 log_stdout(i)，把标准输出重定向到 i.log，并首先输出 factor。

对于后续数字：

- 如果 value % factor 等于 0，则将 value 写入当前日志；
- 如果不能整除，则将 value 通过右侧管道传给下一个子进程；
- 只有第一次遇到需要继续传递的数字时，才创建右侧管道和下一进程，避免创建无用管道；
- 下一进程递归执行 sub_process(p_right, i + 1)。

对应的控制流程如下：

```
main
 |
 ├── 创建第一条管道
 ├── fork 第 0 个子进程
 └── 写入 2～35
          |
          ▼
 sub_process(p, 0)
 factor = 2，输出到 0.log
          |
          ├── 2 的倍数写入 0.log
          └── 其他数字进入下一条管道
                        |
                        ▼
              sub_process(p, 1)
              factor = 3，输出到 1.log
                        |
                        ├── 3 的倍数写入 1.log
                        └── 其他数字继续传递
                                      |
```

```
...
```

每个日志的第一个数字为质数，其余数字是以该质数为最小质因子的合数。例如，0.log 保存 2 的倍数；经过 2 的筛选后，1.log 保存剩余数字中 3 的倍数；2.log 保存剩余数字中 5 的倍数。这样，2 ~ 35 中的每个数字恰好进入一个日志。

所有进程都及时关闭不需要的管道端。当前进程完成写入后关闭右侧写端，使下一级能够收到 EOF，再通过 wait 回收子进程。流水线最终从最右侧开始逐级退出。

在 Makefile 的 UPROGS 中加入：

```
$U/_composites
```

测试结果

composites.c 成功通过 RISC-V GCC 编译和链接，并被写入 fs.img。在 xv6 中执行 composites 后，成功生成 0.log ~ 10.log。

日志文件	文件内容
0.log	2、4、6、8、10、12、14、16、18、20、22、24、26、28、30、32、34
1.log	3、9、15、21、27、33
2.log	5、25、35
3.log	7
4.log	11
5.log	13
6.log	17
7.log	19
8.log	23
9.log	29
10.log	31

测试表明：

1. 2 ~ 35 中的每个数字恰好出现一次；
2. 每个日志的首项依次为 2、3、5、7、11、13、17、19、23、29、31，均为质数；
3. 其余数字均被正确归入最小质因数对应的日志；
4. 连续执行两次 composites 均能正常结束，没有发生管道阻塞或文件描述符耗尽；
5. 执行 composites extra 时能正确输出 Usage: composites。

遇到的问题与解决思路

1. 如果进程保留了不需要的管道写端，下一级 read 会一直等待，不会收到 EOF。解决方法是让每个进程在 fork 后立即关闭无用的读写端，并在数据发送完毕后关闭右侧写端，再调用 wait。
2. GCC 15 在启用 -Werror 时将最初的递归写法判断为可能无限递归。将 sub_process 调整为在输入结束时正常返回，并让每个 fork 出来的子进程在递归调用返回后显式 exit，使递归终止路径更加清晰，之后成功通过编译。

实验三：参数转换 xargs

实现思路

实验三实现简化版 xv6 xargs。程序接收一个待执行命令及其原有参数，从标准输入逐行读取内容，将当前行拆分成附加参数，然后为每一行单独 fork 一个子进程，通过 exec 执行组合后的命令；父进程使用 wait 等待子进程结束后再处理下一行。

例如：

```
echo hello too | xargs echo bye
```

xargs 启动时已有命令和参数：

```
echo bye
```

它从标准输入读到 hello too 后，构造新的参数数组：

```
exec_argv[0] = "echo"
exec_argv[1] = "bye"
exec_argv[2] = "hello"
exec_argv[3] = "too"
exec_argv[4] = 0
```

最终通过 exec 执行：

```
echo bye hello too
```

整体流程如下：

```
标准输入
    |
    | 逐字符读取，直到遇到换行符
    ▼
得到一行文本
    |
    | 按空格、Tab 等空白字符拆分
    ▼
追加到命令原有 argv
    |
    ├── fork 子进程
    |       └── exec(command, exec_argv)
    |
    └── 父进程 wait
```

```
                  |
                  ▼
          继续读取下一行
```

main 使用 read(0, &c, 1) 每次读取一个字符。普通字符被存入长度为 512 的行缓冲区，遇到换行符后调用 run_command；如果输入最后一行没有换行符，则在 EOF 后再处理缓冲区中的剩余内容。程序同时检查读取错误和输入行长度，防止缓冲区越界。

run_command 首先复制 xargs 后原有的命令及参数，然后扫描当前输入行。它跳过空白字符，把参数之间的空白位置改成字符串结束符，并让 exec_argv 中的指针分别指向各参数的起始位置。例如 hello too 在内存中被处理为 hello、结束符、too、结束符，从而不需要额外复制字符串。

参数数组使用 kernel/param.h 中定义的 MAXARG 限制大小，并在最后添加空指针。空行不执行命令；参数过多、fork 失败或 exec 失败时通过标准错误输出提示。

程序在 Makefile 的 UPROGS 中加入：

```
$U/_xargs
```

助教提供的测试脚本还依赖 find 命令，而当前仓库原本没有 user/find.c，因此补充了递归查找程序并在 UPROGS 中加入：

```
$U/_find
```

测试结果

xargs.c 和作为测试依赖的 find.c 均成功通过 RISC-V GCC 编译和链接，并被写入 fs.img。

首先运行助教指定的测试：

```
$ sh < xargstest.sh
$ $ $ $ $ $ hello
hello
hello
$ $
```

脚本创建三个名为 b 且内容为 hello 的文件，再执行：

```
find . b | xargs grep hello
```

find 输出三个文件路径，xargs 为每一行分别执行一次 grep，因此最终出现三次 hello：

```
hello
hello
hello
```

讲义示例测试：

```
$ echo hello too | xargs echo bye
bye hello too
```

多行输入测试使用 xargstest.sh 本身作为输入：

```
$ cat xargstest.sh | xargs echo LINE
LINE mkdir a
LINE echo hello > a/b
LINE mkdir c
LINE echo hello > c/b
LINE echo hello > b
LINE find . b | xargs grep hello
```

输出表明 xargs 对输入的每一行分别执行了一次 echo。

缺少待执行命令时：

```
$ xargs
Usage: xargs command [arguments ...]
```

助教脚本、讲义示例、多行输入和错误参数测试均通过。

遇到的问题与解决思路

1. 第一次运行 xargstest.sh 时出现 exec find failed。检查 fs.img 后发现原仓库没有 find 命令，而测试脚本依赖 find . b。补充 user/find.c、将 $U/_find 加入 UPROGS 并重新生成 fs.img 后，助教脚本成功输出三次 hello。
2. exec 要求 argv 中的每个元素分别指向一个以字符串结束符结尾的参数，并以空指针结束整个数组。解决方法是在原行缓冲区中把参数间的空白字符改成字符串结束符，将各单词起始地址放入 exec_argv，同时利用 MAXARG 检查参数数量并为末尾空指针预留位置。